ASSIGNMENT 13.5

A.Krishna Koushik
2303A51589
BATCH – 19

Task 1
• Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
• Expected Output:
o Refactored version passing rate as a parameter or using a configuration structure.

CODE:

```
class Config:
    def __init__(self, rate):
        self.rate = rate

def calculate_interest(amount, config):
    return amount * config.rate

config = Config(0.1)
print(calculate_interest(1000, config))
```

Observation:
The global variable rate was removed and passed as a parameter to the function. This improves modularity, flexibility, and testability, because the function no longer depends on external variables and can work with different interest rates.

Task 2
• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas:
o Readability

o Logical simplification
o Maintainability
Legacy Code:
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
Expected Outcome:
o Flattened logic using guard clauses or a mapping-based approach.

CODE:

```
score = 78

if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

Observation:
The deeply nested if–else statements were simplified into a cleaner structure. This improves readability and maintainability, making the code easier to understand and modify.
Task 3
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers
o Function reuse
Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
Expected Outcome:
o Reusable function using with open() and parameters.

CODE:

```
def print_file_contents(filename):
    try:
        with open(filename, 'r') as f:
            print(f.read())
    except FileNotFoundError:
        print(f"File '{filename}' not found.")
    except Exception as e:
        print(f"Error reading '{filename}': {e}")

print_file_contents("data1.txt")
print_file_contents("data2.txt")
```

Observation:
The repeated file opening and closing operations were replaced with a reusable function using with open(). This follows the DRY (Don't Repeat Yourself) principle and ensures files are automatically closed, improving code reliability.

Task 4

• Task: Refactor inefficient linear searches using appropriate
data structures.
• Focus Areas:
o Time complexity
o Data structure choice
Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:
o Use of sets or dictionaries with complexity justification.

CODE:

```
def print_file_contents(filename):
    try:
        with open(filename, 'r') as f:
            print(f.read())
    except FileNotFoundError:
        print(f"File '{filename}' not found.")
    except Exception as e:
        print(f"Error reading '{filename}': {e}")

print_file_contents("data1.txt")
```

```
print_file_contents("data2.txt")
```

Observation:
The linear search was replaced with a set-based lookup. Sets provide faster membership checking (O(1) time complexity) compared to lists (O(n)), improving the efficiency of the search operation.

Task 5

• Task: Use AI to refactor procedural code into a class-based design.
• Focus Areas:
o Object-Oriented principles
o Encapsulation
Legacy Code:
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
Expected Outcome:
o A class like EmployeeSalaryCalculator with methods and attributes.

CODE:

```
users = {"admin", "guest", "editor", "viewer"}

name = input("Enter username: ")

if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Observation:
The procedural code was converted into a class-based design using Object-Oriented Programming. This improves encapsulation, reusability, and code organization by grouping related data and methods together.

TASK 6

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:
o Algorithmic optimization
o Use of built-in functions
Legacy Code:
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)
Expected Outcome:
o Optimized logic using mathematical formulas or comprehensions, with time comparison

CODE:

```
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_net_salary(self):
        tax = self.salary * 0.2
        net = self.salary - tax
        return net

emp = EmployeeSalaryCalculator(50000)
print(emp.calculate_net_salary())
```

Observation:

The loop used to calculate the sum of even numbers was optimized using built-in functions or mathematical formulas. This reduces execution time and improves overall performance and efficiency.

Task 7

Task: Refactor code that modifies shared mutable state.
• Focus Areas:
o Functional-style refactoring
o Predictability
Legacy Code:
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
Expected Outcome:
o Refactored function returning values instead of mutating

globals.

CODE:

```
# Legacy code
start = time.time()
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print("Legacy total:", total)
print("Legacy time:", time.time() - start)

# Optimized code using mathematical formula
start = time.time()
# Sum of even numbers from 1 to 999999
# Even numbers: 2, 4, ..., 999998
n = (999998 // 2)  # number of even numbers
optimized_total = n * (2 + 999998) // 2  # arithmetic progression sum
print("Optimized total:", optimized_total)
print("Optimized time:", time.time() - start)

def add_item(data, x):
    return data + [x]

data = []
data = add_item(data, 10)
data = add_item(data, 20)

print(data)
```

Observation:
The function was refactored to return values instead of modifying global variables. This eliminates hidden side effects and makes the function more predictable, reusable, and easier to test.

Task 8
• Task: Use AI to simplify and modularize complex validation rules.
• Focus Areas:
o Readability
o Testability
Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
```

else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
Expected Outcome:
o Separate validation functions with clear responsibility

CODE:

```
def is_long_enough(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

def validate_password(password):
    if not is_long_enough(password):
        return "Password too short"
    if not has_digit(password):
        return "Must contain digit"
    if not has_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"

if __name__ == "__main__":
    password = input("Enter password: ")
    print(validate_password(password))
```

Observation:
The password validation logic was divided into separate validation functions, each responsible for one rule. This improves readability, modularity, and easier testing of validation conditions.